

What is Maven ?

It is a popular automation and dependency management tool used mainly in Java projects.

Why ?

It simplifies the building and management of Java-based projects

Another popular Java project build tool : Gradle

Equivalent build tools

Node : npm / yarn

With npm : package.json : to specify dependencies.

npm install / start options to download n install dependencies n run the app.

dependencies : node_modules

In Java :

Maven : build tool

pom.xml : to specify dependencies + build plugins

Maven goals : clean , install , package ...

In Eclipse IDE (STS) : has built in Maven plug-in (m2e)

Default Maven Home : <user home dir> \.m2

How ?

By providing a structured way to define the project's configuration, dependencies, and build process.

Maven uses a declarative approach, where you define the build plug-ins, dependencies etc n Maven takes care of executing the necessary tasks(goals)

Maven follows a predefined project structure and uses a declarative approach to build and manage Java projects. Here's a general overview of how Maven works:

Maven Project Structure

It has a specific directory structure.

By default,

source code - resides in the src/main/java directory

resources (config xml files) - in the src/main/resources directory.

test code in the src/test/java directory

testing resources : src/test/resources

Maven also expects a pom.xml file in the root directory of your project. This file contains the project's configuration, including dependencies, plugins, and build instructions.

POM (Project Object Model):

The pom.xml file is the heart of the Maven project.

It defines the project's metadata, dependencies, build settings, and more.

The POM contains information such as the project's group ID, artifact ID, version, and packaging type.

It also specifies the project's dependencies, plugins, repositories, and the build process.

Important elements of pom.xml

1. In Maven, the concepts of "Group" and "Artifact" are used to uniquely identify a project and its corresponding artifacts (e.g., JAR files, WAR files).

1.1 Group ID: The group ID is a unique identifier for a project or a logical grouping of related projects. Group IDs typically follow the reverse domain name notation, such as com.developers or org.mydomain or com.app

1.2 Artifact ID: The artifact ID is a unique identifier for a specific artifact (Typically JAR file, WAR file name) produced by a project. It represents the name of the project or module within the group. The artifact ID, along with the group ID, uniquely identifies a specific project or module.

Together, the group ID and artifact ID form the coordinates that identify your project or module within a repository.

2. Dependency Management:

You simply specify the libraries (artifacts) your project depends on, along with their corresponding versions. Maven automatically resolves these dependencies by downloading them from remote repositories or using the local repository. It also resolves transitive dependencies, which are dependencies of your dependencies. Maven ensures that all required dependencies are available before the build process starts.

3. In Maven, repositories are locations where Maven retrieves dependencies (libraries) for your project. A repository can be either local or remote.

Local Repository: The local repository is a directory on your local machine where Maven stores the dependencies it has downloaded. By default, it is located in the .m2 directory under your user's home directory. The local repository serves as a cache, allowing Maven to avoid downloading dependencies repeatedly.

Remote Repository: Remote repositories are located on servers and contain a collection of pre-built libraries (artifacts) that Maven can fetch for your project. These repositories can be public, like Maven Central Repository, or private, hosted within an organization. When Maven encounters a dependency that it needs but doesn't have in the local repository, it searches for it in the remote repositories defined in your project's configuration.

Maven is pre-configured to use the Central Repository as the default remote repository. This repository hosts a vast collection of open-source libraries that can be readily downloaded by Maven. When you specify a dependency in your project's pom.xml file, Maven automatically searches for that dependency in the remote repositories.

On the top of that , you can also add more remote repositories , if needed!

4. Build Life Cycle: Maven defines a build life cycle, which is a predefined sequence of phases and goals that decides the order of execution during the build process.

By default, Maven has three main life cycles: clean, default, and site.

Each life cycle consists of multiple phases, and each phase represents a specific step in the build process. You can execute a specific phase or goal to perform the associated actions, such as compilation, testing, packaging, or deployment.

5. Plug-ins: Maven plug-ins extend its functionality and enable you to perform specific tasks during the build process. Maven provides a wide range of plug-ins for various purposes, such as compiling code, running tests, generating documentation, deploying artifacts, and more. Plug-ins are configured in the pom.xml file, specifying their goals and necessary configuration parameters. Maven resolves and downloads the required plug-ins from repositories to execute the specified goals.

Actual Build Process

When you invoke a Maven build command, such as `mvn clean install`, Maven reads the pom.xml file, initializes the project, and executes the specified goals according to the corresponding life cycle and phase. Maven resolves dependencies, compiles source code, runs tests, packages the project into a distributable format (e.g., JAR, WAR), and performs any additional tasks specified by plug-ins.

Output and Artifacts: Maven generates build outputs and artifacts based on the defined goals and phases.

The artifacts(JAR or WAR file) can be found in the target directory by default.

Maven also generates reports, documentation, and other project-related artifacts based on configured plug-ins and goals.

More Details :

Maven defines a set of goals and a build life cycle to manage the project

Goals: Maven goals are the individual tasks that can be executed during the build process. Each goal performs a specific action and can be executed independently or as part of a larger sequence of goals. Some commonly used Maven goals include clean, compile, test, package, install, and deploy. For example, compile compiles the project's source code, test runs the project's tests, and package creates a distributable package.

Build Life Cycle: The Maven build life cycle is a predefined sequence of phases that dictate the order in which goals are executed. The life cycle consists of several phases, and each phase represents a distinct stage in the build process. The common Maven life cycle consists of three main parts: clean, default, and site.

Clean Life Cycle: The clean life cycle handles the cleaning of artifacts generated by previous builds. It has a single phase called clean, which removes the build output directory (target by default) to ensure a clean build environment.

Default Life Cycle: The default life cycle is the core of the Maven build process. It handles the compilation, testing, packaging, and deployment of your project. It comprises the following phases (among others):

validate: Validates the project's structure and configuration.

compile: Compiles the project's source code into bytecode.

test: Executes unit tests for the project.

package: Packages compiled code and resources into a distributable format (e.g., JAR, WAR).

install: Installs the packaged artifact into the local Maven repository for use in other projects.

deploy: Copies the artifact to a remote repository for sharing with other developers or projects.

Site Life Cycle: The site life cycle generates project documentation and reports. It includes phases such as site, site-deploy, and site:stage.

By invoking a specific Maven goal, you trigger the associated life cycle and all the preceding phases up to

the invoked goal. Maven determines the appropriate phases to execute based on the goal you specify.

Maven's life cycle and goals provide a standardized and repeatable way to build and manage Java projects, promoting consistency across development teams and facilitating project collaboration.